

Documento de diseño de software

Proyecto: Cooperativa Financiera El Progreso — sistema de ventanilla para asociados

Norma: 220501095 — Diseñar la solución de software de acuerdo con procedimientos y requisitos técnicos

1. Introducción del sistema

La Cooperativa Financiera El Progreso administra los ahorros de aproximadamente 300 asociados. Hoy ese registro se lleva en un cuaderno de ventanilla y en Excel, lo que ya generó un incidente real: una cajera dejó a un asociado retirar más dinero del que tenía, dejando el saldo en negativo. Adicionalmente, calcular el saldo en dólares o producir un informe para el consejo directivo exige trabajo manual sobre el Excel cada vez que se necesita.

El sistema a diseñar es una aplicación de consola de uso exclusivo de la cajera (el asociado nunca opera el sistema directamente) que centraliza el registro de asociados y sus movimientos de ahorro, aplica las reglas de negocio de forma automática e inquebrantable (nunca se puede dejar un saldo en negativo, ni omitir una comisión), y entrega informes gerenciales que hoy toma horas armar a mano.

2. Objetivos del sistema

- Eliminar el error humano en el cálculo de saldos: el saldo nunca se digita, siempre se deriva de los movimientos registrados.
- Permitir que la cajera ubique a un asociado de inmediato, ya sea por número de documento o por nombre (búsqueda parcial, sin distinguir mayúsculas/minúsculas).
- Aplicar automáticamente la comisión de manejo de efectivo en retiros grandes, sin que dependa de que la cajera se acuerde de cobrarla.

- Ofrecer trazabilidad total: reconstruir todo el historial de movimientos de un asociado ante cualquier reclamo.
- Convertir el saldo a dólares usando la Tasa Representativa del Mercado (TRM) oficial, sin que la cajera tenga que buscarla manualmente, e indicando el periodo de vigencia de la tasa usada.
- Producir en segundos los seis informes que la gerencia necesita para tomar decisiones, sin filtrar Excel a mano.

3. Actores / usuarios del sistema

Actor	Rol
Cajera	Actor principal y único que opera el sistema. Registra asociados, recibe/entrega dinero, consulta saldos e informes.
Gerente (Don Rafael)	Usuario indirecto: no opera el sistema él mismo, pero consume los informes de gerencia que la cajera consulta por él.
Asociado	No toca el sistema. Se identifica ante la cajera con su número de documento; el sistema solo lo representa como datos.
API de la TRM (Superintendencia Financiera / datos.gov.co)	Sistema externo consultado de forma asíncrona para obtener la tasa de cambio oficial. Es un actor no humano ("sistema externo").

4. Requisitos funcionales

Extraídos directamente de lo que Don Rafael pidió; cada uno corresponde a una opción del menú principal:

- 1) Registrar un asociado nuevo (queda con saldo en \$0).
- 2) Listar todos los asociados.

- 3) Buscar un asociado por número de documento.
- 4) Buscar un asociado por nombre (coincidencia parcial, sin distinguir mayúsculas/minúsculas).
- 5) Actualizar los datos de un asociado (nombre, teléfono, dirección — el documento no cambia, es su identificador).
- 6) Eliminar un asociado (solo si no tiene saldo ni movimientos registrados).
- 7) Consultar el saldo de un asociado en pesos.
- 8) Consultar el saldo de un asociado convertido a dólares con la TRM oficial, indicando el periodo de vigencia de la tasa.
- 9) Registrar una consignación.
- 10) Registrar un retiro (con comisión automática si aplica, y rechazo si deja el saldo en negativo).
- 11) Ver todos los movimientos de un asociado, con fecha, tipo y valor.
- 12) Consultar los seis informes de gerencia (resumen general, top 5 por saldo, asociados dormidos, informe por periodo, movimientos más grandes, actividad por asociado).

5. Requisitos no funcionales

Requisito	Cómo se atiende en el diseño
Disponibilidad ante fallas externas	Si la API de la TRM no responde (caída, timeout, JSON inválido), el sistema no se cae: se informa a la cajera y sigue operando con normalidad.
Consumo asíncrono de la TRM	La consulta a la API se hace con HttpClient de forma asíncrona (async/await), sin bloquear el hilo de la consola mientras se espera la respuesta.
Consistencia de datos	El saldo jamás es un campo editable directamente; siempre se calcula a partir del historial de movimientos, así

	se elimina la clase de error que causó el incidente del retiro en negativo.
Usabilidad para personal no técnico	Todos los textos que ve la cajera están en español, el menú es numérico y cerrado (no acepta texto libre para navegar), y cada entrada inválida se vuelve a pedir con un mensaje claro en vez de tumbar el programa.
Trazabilidad / auditoría	Todo movimiento aceptado queda registrado con tipo, valor y fecha exacta; un movimiento rechazado no se ejecuta ni se registra, y el saldo queda intacto.
Mantenibilidad / extensibilidad	Arquitectura en capas con persistencia detrás de una interfaz (IMemberRepository), de modo que cambiar el almacenamiento en memoria por una base de datos no obliga a tocar la lógica de negocio ni la interfaz de usuario.
Rendimiento	Con ~300 asociados y sus movimientos, una lista en memoria y LINQ son más que suficientes; no se requiere indexación ni caché.

6. Arquitectura de la solución

Arquitectura en capas, separadas físicamente por carpeta dentro de un único proyecto de consola (.NET 10 / C# 14). Cada capa solo puede depender de la(s) de más abajo:

UI (TellerConsole, ...)	← única capa que usa Console.
Services (reglas de uso)	← MemberService , MovementService , ReportService , ExchangeRateService .
Repositories (persistencia)	← IMemberRepository + InMemoryMemberRepository .
Models (entidades + reglas de negocio intrínsecas)	← Member , Movement , MovementType .

Program.cs actúa como *composition root*: ahí se instancian las clases concretas (**InMemoryMemberRepository**, **HttpClient**, los **Service**) y se inyectan por constructor en **TellerConsole**. Ninguna capa superior sabe cómo está implementada la de abajo — solo conoce sus contratos (interfaces o firmas de métodos), lo que permite sustituir piezas (por ejemplo, la persistencia) sin reescribir el resto del sistema. El diagrama de clases (evidencia 2) detalla las relaciones exactas.

7. Descripción de módulos

Módulo (carpeta)	Responsabilidad	Clases principales
Models	Representar al asociado y sus movimientos, y contener las reglas de negocio que nunca deben poder saltarse (saldo derivado,	Member , Movement , MovementType .

	validaciones, comisión).	
Repositories	Guardar y recuperar asociados, sin que quien los use sepa cómo se almacenan.	IMemberRepository, InMemoryMemberRepository.
Services	Casos de uso: orquestar repositorio + modelo para cumplir una operación completa (registrar, consultar, listar informes, convertir a USD).	MemberService, MovementService, ReportService, ExchangeRateService.
UI	Interacción con la cajera por consola: mostrar menús, leer y validar entradas, mostrar resultados y mensajes de error.	TellerConsole, ManagementReportsMenu, ConsoleReader, ConsoleFormat.
Program.cs	Punto de entrada; arma el grafo de dependencias y arranca el menú.	—

8. Tecnologías a utilizar

- .NET 10 / C# 14 — última versión LTS del framework al momento del desarrollo; se usan *primary constructors*, *collection expressions* (`[]`) y *raw string literals* para un código más compacto y legible.

- **System.Net.Http.HttpClient** — consumo asíncrono de la API pública de la TRM (**datos.gov.co**), sin necesidad de librerías externas de terceros.
- **System.Text.Json** — deserialización de la respuesta JSON de la TRM (**GetFromJsonAsync<T>**), incluida en el framework.
- LINQ — consultas declarativas para búsquedas, ordenamientos y agregaciones en los informes gerenciales (**Where**, **OrderBy**, **Sum**, **Take**, **GroupBy**-equivalentes).
- **System.Text.RegularExpressions** (Source Generators) — validación de formato de documento, nombre y teléfono con **[GeneratedRegex]**, que compila la expresión regular en tiempo de compilación en vez de interpretarla en cada ejecución.
- Sin base de datos ni ORM en esta entrega (ver justificación técnica), y sin frameworks de UI: consola pura (**System.Console**).

9. Justificación técnica

- **Arquitectura en capas por carpeta, no por proyecto:** con un solo consumidor (la consola) y un alcance de ejercicio acotado, separar en múltiples proyectos .csproj habría añadido complejidad de compilación sin beneficio real. La separación por carpeta ya impone los mismos límites de dependencia (verificables por revisión de **using**), y se puede migrar a proyectos separados después si el sistema crece.
- **IMemberRepository como interfaz:** es la decisión que hace el diseño extensible. **MemberService** y **MovementService** reciben la interfaz por constructor, nunca la clase concreta. El día de mañana, sustituir **InMemoryMemberRepository** por una implementación con Entity Framework o Dapper es un cambio confinado a **Program.cs** y a una clase nueva — cero cambios en reglas de negocio o en la consola.
- **El saldo como propiedad calculada, nunca como campo:** **Member.Balance** es **_movements.Sum(m => m.SignedEffect)**. No existe ningún setter público de saldo en todo el sistema. Esto ataca directamente la causa raíz del incidente que motivó el proyecto (un retiro que dejó el saldo

en negativo) al nivel del modelo de datos, no como una validación que alguien podría olvidar poner en la capa de UI.

- **Persistencia en memoria en vez de base de datos:** el enunciado no pide persistencia entre ejecuciones, y una lista en memoria detrás de **IMemberRepository** permite entregar las reglas de negocio correctas dentro del tiempo del ejercicio sin sacrificar la posibilidad de agregar persistencia real después (ver evidencia 4, modelo de base de datos, que sí queda diseñado para ese paso siguiente).
- **Movement como record inmutable:** un movimiento contable, una vez registrado, no debe poder mutar. Un **record** da igualdad por valor y inmutabilidad "gratis" del lenguaje, comunicando esa intención sin código adicional.
- **Excepciones tipadas (ArgumentException / InvalidOperationException) en vez de un tipo Result:** con una sola capa de UI consumidora, el costo de introducir un tipo **Result<T>** no se justificaba; las excepciones ya traen en **Message** el texto exacto que debe leer la cajera, y **TellerConsole** las captura por cada opción del menú.
- **Comisión de manejo de efectivo integrada en RegisterWithdrawal, no en la UI:** si el cálculo de la comisión viviera en la consola, cualquier otro punto de entrada futuro (una API, por ejemplo) podría olvidarla. Al vivir dentro de **Member**, es físicamente imposible retirar dinero sin pasar por esa regla.